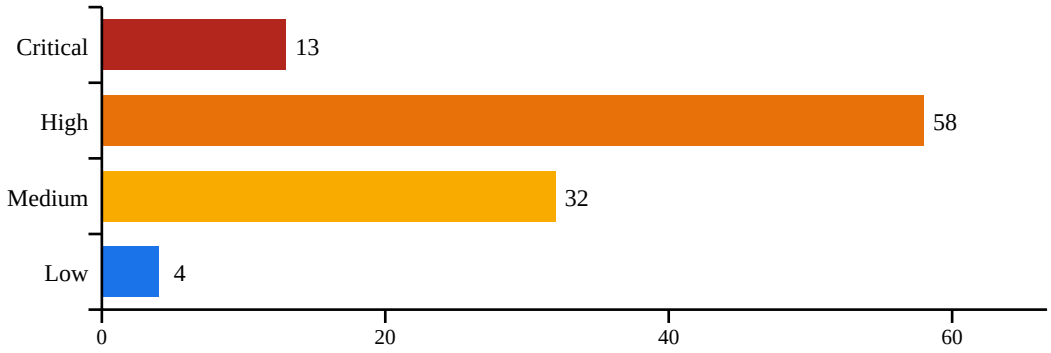
CodeMender Security Report

https://github.com/juice-shop/juice-shop @ 399fba7 · 2026-10-22 08:55 UTC · Engine: Built-in rule engine (CodeMender-style)

100	107	19	615
Risk score / 100	Findings	With ready patch	Files · 137,028 lines

Findings by severity



OWASP Top 10 (2021) exposure

Category	Findings
A03:2021 Injection	42
A05:2021 Security Misconfiguration	30
A07:2021 Identification and Authentication Failures	20
A01:2021 Broken Access Control	8
A09:2021 Security Logging and Monitoring Failures	4
A02:2021 Cryptographic Failures	2
A10:2021 Server-Side Request Forgery	1

Findings overview

Sev	ID	Issue	CWE	Location	Status
Critical	CM-860943e8	SQL injection via string-built query	CWE-89	data/static/codefixes/dbSchemaChallenge_1.ts:5	-
Critical	CM-1abd44eb	SQL injection via string-built query	CWE-89	data/static/codefixes/dbSchemaChallenge_3.ts:11	-
Critical	CM-25281010	SQL injection via string-built query	CWE-89	data/static/codefixes/loginAdminChallenge_1.ts:18	-
Critical	CM-36c49330	SQL injection via string-built query	CWE-89	data/static/codefixes/loginAdminChallenge_2.ts:15	-
Critical	CM-741df378	SQL injection via string-built query	CWE-89	data/static/codefixes/loginBenderChallenge_1.ts:18	-
Critical	CM-057bd162	SQL injection via string-built query	CWE-89	data/static/codefixes/loginBenderChallenge_3.ts:15	-
Critical	CM-6fdffff9	SQL injection via string-built query	CWE-89	data/static/codefixes/loginBenderChallenge_4.ts:15	-

Sev	ID	Issue	CWE	Location	Status
Critical	CM-b7a40b7e	SQL injection via string-built query	CWE-89	data/static/codefixes/loginJimChallenge_2.ts:15	-
Critical	CM-628cd131	SQL injection via string-built query	CWE-89	data/static/codefixes/loginJimChallenge_4.ts:18	-
Critical	CM-f4e1b7b0	SQL injection via string-built query	CWE-89	data/static/codefixes/unionSqlInjectionChallenge_1.ts:6	-
Critical	CM-fc0b37f2	SQL injection via string-built query	CWE-89	data/static/codefixes/unionSqlInjectionChallenge_3.ts:10	-
Critical	CM-1225de7f	SQL injection via string-built query	CWE-89	routes/login.ts:34	-
Critical	CM-2d2abbfe	SQL injection via string-built query	CWE-89	routes/search.ts:23	-
High	CM-a52a4e15	NoSQL / query operator injection	CWE-943	data/static/codefixes/chatbotGreedyInjectionChallenge_1.ts:65	-
High	CM-e5f9169d	NoSQL / query operator injection	CWE-943	data/static/codefixes/chatbotGreedyInjectionChallenge_2_correct.ts:66	-
High	CM-e400e596	NoSQL / query operator injection	CWE-943	data/static/codefixes/chatbotGreedyInjectionChallenge_3.ts:66	-
High	CM-12309412	NoSQL / query operator injection	CWE-943	data/static/codefixes/chatbotGreedyInjectionChallenge_4.ts:66	-
High	CM-fc1f8c86	Hard-coded credential / API key	CWE-798	data/static/users.yml:3	-
High	CM-a332d6d8	Hard-coded credential / API key	CWE-798	data/static/users.yml:31	-
High	CM-9f9f7167	Hard-coded credential / API key	CWE-798	data/static/users.yml:63	-
High	CM-7643de05	Hard-coded credential / API key	CWE-798	data/static/users.yml:88	-
High	CM-98bde1d7	Hard-coded credential / API key	CWE-798	data/static/users.yml:107	-
High	CM-087b2fee	Hard-coded credential / API key	CWE-798	data/static/users.yml:115	-
High	CM-d65f2f02	Hard-coded credential / API key	CWE-798	data/static/users.yml:123	-
High	CM-5e690dc0	Hard-coded credential / API key	CWE-798	data/static/users.yml:141	-
High	CM-ad186add	Hard-coded credential / API key	CWE-798	data/static/users.yml:150	-
High	CM-eb0ede55	Hard-coded credential / API key	CWE-798	data/static/users.yml:159	-
High	CM-9c40940e	Hard-coded credential / API key	CWE-798	data/static/users.yml:239	-
High	CM-5154ca0b	Hard-coded credential / API key	CWE-798	data/static/users.yml:245	-
High	CM-04335010	Hard-coded credential / API key	CWE-798	data/static/users.yml:261	-
High	CM-5b43abe4	Hard-coded credential / API key	CWE-798	data/static/users.yml:272	-
High	CM-7bfbfb15	Hard-coded credential / API key	CWE-798	data/static/users.yml:277	-
High	CM-9197a472	Cross-site scripting (unsafe HTML sink)	CWE-79	frontend/src/app/data-export/data-export.component.ts:72	-
High	CM-25a30f61	Hard-coded credential / API key	CWE-798	frontend/src/app/faucet/faucet.component.ts:34	-
High	CM-ecf02202	Hard-coded credential / API key	CWE-798	frontend/src/app/login/login.component.ts:69	-
High	CM-1de1e6a2	Cross-site scripting (unsafe HTML sink)	CWE-79	frontend/src/assets/private/dat.gui.min.js:1	patch
High	CM-437d5842	Cross-site scripting (unsafe HTML sink)	CWE-79	frontend/src/assets/private/dat.gui.min.js:13	patch
High	CM-7cef06e1	Cross-site scripting (unsafe HTML sink)	CWE-79	frontend/src/assets/private/dat.gui.min.js:23	patch
High	CM-9cb42b4f	Cross-site scripting (unsafe HTML sink)	CWE-79	frontend/src/assets/private/dat.gui.min.js:33	patch
High	CM-c8a83128	Cross-site scripting (unsafe HTML sink)	CWE-79	frontend/src/assets/private/dat.gui.min.js:35	patch

Sev	ID	Issue	CWE	Location	Status
High	CM-1652ac93	Cross-site scripting (unsafe HTML sink)	CWE-79	frontend/src/assets/private/dat.gui.min.js:39	patch
High	CM-9bf87dea	Cross-site scripting (unsafe HTML sink)	CWE-79	frontend/src/assets/private/dat.gui.min.js:40	patch
High	CM-e72e9ced	Cross-site scripting (unsafe HTML sink)	CWE-79	frontend/src/assets/private/dat.gui.min.js:41	patch
High	CM-1bba65f6	Cross-site scripting (unsafe HTML sink)	CWE-79	frontend/src/assets/private/dat.gui.min.js:44	patch
High	CM-84b26eee	Cross-site scripting (unsafe HTML sink)	CWE-79	frontend/src/assets/private/dat.gui.min.js:48	patch
High	CM-d5d059de	Cross-site scripting (unsafe HTML sink)	CWE-79	frontend/src/assets/private/dat.gui.min.js:49	patch
High	CM-765d9f9a	Cross-site scripting (unsafe HTML sink)	CWE-79	frontend/src/assets/private/dat.gui.min.js:55	patch
High	CM-793c7087	Cross-site scripting (unsafe HTML sink)	CWE-79	frontend/src/assets/private/stats.min.js:3	patch
High	CM-374c7e1b	Cross-site scripting (unsafe HTML sink)	CWE-79	frontend/src/assets/private/stats.min.js:4	patch
High	CM-91f910d4	Cross-site scripting (unsafe HTML sink)	CWE-79	frontend/src/assets/private/three.js:11354	patch
High	CM-8d16daed	Cross-site scripting (unsafe HTML sink)	CWE-79	frontend/src/assets/private/three.js:11375	patch
High	CM-37c445b6	Cross-site scripting (unsafe HTML sink)	CWE-79	frontend/src/hacking-instructor/helpers/helpers.ts:138	patch
High	CM-8397a5f0	Cross-site scripting (unsafe HTML sink)	CWE-79	frontend/src/hacking-instructor/index.ts:126	patch
High	CM-b69f5120	Cross-site scripting (unsafe HTML sink)	CWE-79	frontend/src/hacking-instructor/index.ts:142	patch
High	CM-57da77f4	Hard-coded credential / API key	CWE-798	infrastructure/terraform/networking.tf:171	-
High	CM-a292874b	Hard-coded credential / API key	CWE-798	lib/insecurity.ts:21	-
High	CM-85868b16	Code injection via eval/exec	CWE-95	routes/captcha.ts:22	-
High	CM-933f1b2a	NoSQL / query operator injection	CWE-943	routes/chat.ts:149	-
High	CM-8a7e6bc3	Path traversal in file access	CWE-22	routes/languages.ts:31	-
High	CM-ba6c8c25	Path traversal in file access	CWE-22	routes/languages.ts:40	-
High	CM-a69146ae	Server-side request forgery (SSRF)	CWE-918	routes/profileImageUrlUpload.ts:24	-
High	CM-2224f7a2	NoSQL / query operator injection	CWE-943	routes/showProductReviews.ts:36	-
High	CM-0e43a84e	NoSQL / query operator injection	CWE-943	routes/trackOrder.ts:18	-
High	CM-5f8af125	Code injection via eval/exec	CWE-95	routes/userProfile.ts:65	-
High	CM-eff5f38f	Path traversal in file access	CWE-22	routes/videoHandler.ts:82	-
High	CM-6f20c7f2	Path traversal in file access	CWE-22	routes/vulnCodeFixes.ts:30	-
High	CM-0505e1bc	Path traversal in file access	CWE-22	routes/vulnCodeFixes.ts:83	-
High	CM-5902c2e8	Path traversal in file access	CWE-22	routes/vulnCodeSnippet.ts:90	-
High	CM-b566d44c	Path traversal in file access	CWE-22	rsn/rsnUtil.ts:66	-
High	CM-356c117b	Path traversal in file access	CWE-22	rsn/rsnUtil.ts:155	-
High	CM-52ab8096	Hard-coded credential / API key	CWE-798	terraform/networking.tf:171	-
Medium	CM-bcb54971	Container / infra runs privileged or as root	CWE-250	data/static/codefixes/iacLeakedKeyChallenge_1.tf:45	-

Sev	ID	Issue	CWE	Location	Status
Medium	CM-d9585a59	Container / infra runs privileged or as root	CWE-250	data/static/codefixes/iacLeakedKeyChallenge_1.tf:71	-
Medium	CM-63f5604f	Container / infra runs privileged or as root	CWE-250	data/static/codefixes/iacLeakedKeyChallenge_1.tf:78	-
Medium	CM-a4206485	Container / infra runs privileged or as root	CWE-250	data/static/codefixes/iacLeakedKeyChallenge_1.tf:85	-
Medium	CM-ee777e04	Container / infra runs privileged or as root	CWE-250	data/static/codefixes/iacLeakedKeyChallenge_1.tf:111	-
Medium	CM-f20f8bd7	Container / infra runs privileged or as root	CWE-250	data/static/codefixes/iacLeakedKeyChallenge_1.tf:238	-
Medium	CM-46622192	Container / infra runs privileged or as root	CWE-250	data/static/codefixes/iacLeakedKeyChallenge_2.tf:45	-
Medium	CM-24f82af6	Container / infra runs privileged or as root	CWE-250	data/static/codefixes/iacLeakedKeyChallenge_2.tf:71	-
Medium	CM-7e4adf22	Container / infra runs privileged or as root	CWE-250	data/static/codefixes/iacLeakedKeyChallenge_2.tf:78	-
Medium	CM-28881df5	Container / infra runs privileged or as root	CWE-250	data/static/codefixes/iacLeakedKeyChallenge_2.tf:85	-
Medium	CM-3455ce09	Container / infra runs privileged or as root	CWE-250	data/static/codefixes/iacLeakedKeyChallenge_2.tf:111	-
Medium	CM-b0348e25	Container / infra runs privileged or as root	CWE-250	data/static/codefixes/iacLeakedKeyChallenge_2.tf:238	-
Medium	CM-2a84d499	Container / infra runs privileged or as root	CWE-250	data/static/codefixes/iacLeakedKeyChallenge_3_correct.tf:45	-
Medium	CM-2805f9ef	Container / infra runs privileged or as root	CWE-250	data/static/codefixes/iacLeakedKeyChallenge_3_correct.tf:71	-
Medium	CM-9a77da70	Container / infra runs privileged or as root	CWE-250	data/static/codefixes/iacLeakedKeyChallenge_3_correct.tf:78	-
Medium	CM-8bcb68e8	Container / infra runs privileged or as root	CWE-250	data/static/codefixes/iacLeakedKeyChallenge_3_correct.tf:85	-
Medium	CM-4a8670ec	Container / infra runs privileged or as root	CWE-250	data/static/codefixes/iacLeakedKeyChallenge_3_correct.tf:111	-
Medium	CM-c6f453b2	Container / infra runs privileged or as root	CWE-250	data/static/codefixes/iacLeakedKeyChallenge_3_correct.tf:238	-
Medium	CM-4900e461	Container / infra runs privileged or as root	CWE-250	infrastructure/terraform/networking.tf:46	-
Medium	CM-b27b70e3	Container / infra runs privileged or as root	CWE-250	infrastructure/terraform/networking.tf:72	-
Medium	CM-23874a0f	Container / infra runs privileged or as root	CWE-250	infrastructure/terraform/networking.tf:79	-
Medium	CM-ac2233b8	Container / infra runs privileged or as root	CWE-250	infrastructure/terraform/networking.tf:86	-
Medium	CM-40956659	Container / infra runs privileged or as root	CWE-250	infrastructure/terraform/networking.tf:112	-
Medium	CM-35d54dc9	Container / infra runs privileged or as root	CWE-250	infrastructure/terraform/networking.tf:239	-
Medium	CM-506d3251	Weak hash algorithm (MD5/SHA-1)	CWE-328	lib/insecurity.ts:41	-
Medium	CM-74ac5339	Weak hash algorithm (MD5/SHA-1)	CWE-328	scripts/package.mjs:121	-
Medium	CM-405b09ba	Container / infra runs privileged or as root	CWE-250	terraform/networking.tf:46	-
Medium	CM-5ea582e0	Container / infra runs privileged or as root	CWE-250	terraform/networking.tf:72	-
Medium	CM-e150d11b	Container / infra runs privileged or as root	CWE-250	terraform/networking.tf:79	-

Sev	ID	Issue	CWE	Location	Status
Medium	CM-65e255b1	Container / infra runs privileged or as root	CWE-250	terraform/networking.tf:86	-
Medium	CM-b450d24c	Container / infra runs privileged or as root	CWE-250	terraform/networking.tf:112	-
Medium	CM-4af9a423	Container / infra runs privileged or as root	CWE-250	terraform/networking.tf:239	-
Low	CM-56d2ca0f	Sensitive data written to logs	CWE-532	data/datacreator.ts:278	-
Low	CM-dcfaa6eb	Sensitive data written to logs	CWE-532	frontend/src/app/faucet/faucet.component.ts:239	-
Low	CM-3a0140e3	Sensitive data written to logs	CWE-532	frontend/src/hacking-instructor/helpers/helpers.ts:191	-
Low	CM-a673c1ce	Sensitive data written to logs	CWE-532	routes/chat.ts:255	-

Detailed findings

[CRITICAL] SQL injection via string-built query · CWE-89 · A03:2021 Injection

data/static/codefixes/dbSchemaChallenge_1.ts:5 · CM-860943e8 · confidence HIGH
```
models.sequelize.query("SELECT * FROM Products WHERE ((name LIKE '%"+criteria+"%' OR description LIKE '%"+criteria+"%')
```

Why it matters: User-influenced data is concatenated/interpolated into a SQL statement, allowing query manipulation.

Recommended fix: Use parameterised queries / prepared statements; never build SQL with string formatting.

Verification plan: Send a quote-breaking value to the affected parameter in an isolated test DB and confirm the query structure changes or errors.

[CRITICAL] SQL injection via string-built query · CWE-89 · A03:2021 Injection

data/static/codefixes/dbSchemaChallenge_3.ts:11 · CM-1abd44eb · confidence HIGH
```
models.sequelize.query(`SELECT * FROM Products WHERE ((name LIKE '%${criteria}%' OR description LIKE '%${criteria}%') A
```

Why it matters: User-influenced data is concatenated/interpolated into a SQL statement, allowing query manipulation.

Recommended fix: Use parameterised queries / prepared statements; never build SQL with string formatting.

Verification plan: Send a quote-breaking value to the affected parameter in an isolated test DB and confirm the query structure changes or errors.

[CRITICAL] SQL injection via string-built query · CWE-89 · A03:2021 Injection

data/static/codefixes/loginAdminChallenge_1.ts:18 · CM-25281010 · confidence HIGH
```
models.sequelize.query(`SELECT * FROM Users WHERE email = '${req.body.email || ''}' AND password = '${security.hash(req
```

Why it matters: User-influenced data is concatenated/interpolated into a SQL statement, allowing query manipulation.

Recommended fix: Use parameterised queries / prepared statements; never build SQL with string formatting.

Verification plan: Send a quote-breaking value to the affected parameter in an isolated test DB and confirm the query structure changes or errors.

[CRITICAL] SQL injection via string-built query · CWE-89 · A03:2021 Injection

data/static/codefixes/loginAdminChallenge_2.ts:15 · CM-36c49330 · confidence HIGH
```
models.sequelize.query(`SELECT * FROM Users WHERE email = $1 AND password = '${security.hash(req.body.password || '')}'
```

Why it matters: User-influenced data is concatenated/interpolated into a SQL statement, allowing query manipulation.

Recommended fix: Use parameterised queries / prepared statements; never build SQL with string formatting.

Verification plan: Send a quote-breaking value to the affected parameter in an isolated test DB and confirm the query structure changes or errors.

[CRITICAL] SQL injection via string-built query · CWE-89 · A03:2021 Injection

data/static/codefixes/loginBenderChallenge_1.ts:18 · CM-741df378 · confidence HIGH
```
models.sequelize.query(`SELECT * FROM Users WHERE email = '${req.body.email || ''}' AND password = '${security.hash(req
```

Why it matters: User-influenced data is concatenated/interpolated into a SQL statement, allowing query manipulation.

Recommended fix: Use parameterised queries / prepared statements; never build SQL with string formatting.

Verification plan: Send a quote-breaking value to the affected parameter in an isolated test DB and confirm the query structure changes or errors.

[CRITICAL] SQL injection via string-built query · CWE-89 · A03:2021 Injection

data/static/codefixes/loginBenderChallenge_3.ts:15 · CM-057bd162 · confidence HIGH
```
models.sequelize.query(`SELECT * FROM Users WHERE email = :mail AND password = '${security.hash(req.body.password || ''
```

Why it matters: User-influenced data is concatenated/interpolated into a SQL statement, allowing query manipulation.

Recommended fix: Use parameterised queries / prepared statements; never build SQL with string formatting.

Verification plan: Send a quote-breaking value to the affected parameter in an isolated test DB and confirm the query structure changes or errors.

[CRITICAL] SQL injection via string-built query · CWE-89 · A03:2021 Injection

data/static/codefixes/loginBenderChallenge_4.ts:15 · CM-6fdffff9 · confidence HIGH
```
models.sequelize.query(`SELECT * FROM Users WHERE email = '${req.body.email || ''}' AND password = '${security.hash(req
```

Why it matters: User-influenced data is concatenated/interpolated into a SQL statement, allowing query manipulation.

Recommended fix: Use parameterised queries / prepared statements; never build SQL with string formatting.

Verification plan: Send a quote-breaking value to the affected parameter in an isolated test DB and confirm the query structure changes or errors.

[CRITICAL] SQL injection via string-built query · CWE-89 · A03:2021 Injection

data/static/codefixes/loginJimChallenge_2.ts:15 · CM-b7a40b7e · confidence HIGH
```
models.sequelize.query(`SELECT * FROM Users WHERE email = '${req.body.email || ''}' AND password = '${security.hash(req
```

Why it matters: User-influenced data is concatenated/interpolated into a SQL statement, allowing query manipulation.

Recommended fix: Use parameterised queries / prepared statements; never build SQL with string formatting.

Verification plan: Send a quote-breaking value to the affected parameter in an isolated test DB and confirm the query structure changes or errors.

[CRITICAL] SQL injection via string-built query · CWE-89 · A03:2021 Injection

data/static/codefixes/loginJimChallenge_4.ts:18 · CM-628cd131 · confidence HIGH
```
models.sequelize.query(`SELECT * FROM Users WHERE email = '${req.body.email || ''}' AND password = '${security.hash(req
```

Why it matters: User-influenced data is concatenated/interpolated into a SQL statement, allowing query manipulation.

Recommended fix: Use parameterised queries / prepared statements; never build SQL with string formatting.

Verification plan: Send a quote-breaking value to the affected parameter in an isolated test DB and confirm the query structure changes or errors.

[CRITICAL] SQL injection via string-built query · CWE-89 · A03:2021 Injection

data/static/codefixes/unionSqlInjectionChallenge_1.ts:6 · CM-f4e1b7b0 · confidence HIGH
```
models.sequelize.query(`SELECT * FROM Products WHERE ((name LIKE '%${criteria}%' OR description LIKE '%${criteria}%') A
```

Why it matters: User-influenced data is concatenated/interpolated into a SQL statement, allowing query manipulation.

Recommended fix: Use parameterised queries / prepared statements; never build SQL with string formatting.

Verification plan: Send a quote-breaking value to the affected parameter in an isolated test DB and confirm the query structure changes or errors.

[CRITICAL] SQL injection via string-built query · CWE-89 · A03:2021 Injection

data/static/codefixes/unionSqlInjectionChallenge_3.ts:10 · CM-fc0b37f2 · confidence HIGH
```
models.sequelize.query(`SELECT * FROM Products WHERE ((name LIKE '%${criteria}%' OR description LIKE '%${criteria}%') A
```

Why it matters: User-influenced data is concatenated/interpolated into a SQL statement, allowing query manipulation.

Recommended fix: Use parameterised queries / prepared statements; never build SQL with string formatting.

Verification plan: Send a quote-breaking value to the affected parameter in an isolated test DB and confirm the query structure changes or errors.

[CRITICAL] SQL injection via string-built query · CWE-89 · A03:2021 Injection

routes/login.ts:34 · CM-1225de7f · confidence HIGH
```
models.sequelize.query(`SELECT * FROM Users WHERE email = '${req.body.email || ''}' AND password = '${security.hash(req
```

Why it matters: User-influenced data is concatenated/interpolated into a SQL statement, allowing query manipulation.

Recommended fix: Use parameterised queries / prepared statements; never build SQL with string formatting.

Verification plan: Send a quote-breaking value to the affected parameter in an isolated test DB and confirm the query structure changes or errors.

[CRITICAL] SQL injection via string-built query · CWE-89 · A03:2021 Injection

routes/search.ts:23 · CM-2d2abbfe · confidence HIGH
```
models.sequelize.query(`SELECT * FROM Products WHERE ((name LIKE '%${criteria}%' OR description LIKE '%${criteria}%') A
```

Why it matters: User-influenced data is concatenated/interpolated into a SQL statement, allowing query manipulation.

Recommended fix: Use parameterised queries / prepared statements; never build SQL with string formatting.

Verification plan: Send a quote-breaking value to the affected parameter in an isolated test DB and confirm the query structure changes or errors.

[HIGH] NoSQL / query operator injection · CWE-943 · A03:2021 Injection

data/static/codefixes/chatbotGreedyInjectionChallenge_1.ts:65 · CM-a52a4e15 · confidence MEDIUM
```
return await db.reviewsCollection.find({ $where: 'this.product == ' + productId }) as Review[]
```

Why it matters: Request objects are passed directly to a database query, allowing operator injection ($ne, $gt, $where).

Recommended fix: Validate and cast types explicitly; reject objects where scalars are expected.

Verification plan: Send {"$ne": null} as a credential field in a test DB and confirm authentication bypass.

[HIGH] NoSQL / query operator injection · CWE-943 · A03:2021 Injection

data/static/codefixes/chatbotGreedyInjectionChallenge_2_correct.ts:66 · CM-e5f9169d · confidence MEDIUM
```
return await db.reviewsCollection.find({ $where: 'this.product == ' + productId }) as Review[]
```

Why it matters: Request objects are passed directly to a database query, allowing operator injection ($ne, $gt, $where).

Recommended fix: Validate and cast types explicitly; reject objects where scalars are expected.

Verification plan: Send {"$ne": null} as a credential field in a test DB and confirm authentication bypass.

[HIGH] NoSQL / query operator injection · CWE-943 · A03:2021 Injection

data/static/codefixes/chatbotGreedyInjectionChallenge_3.ts:66 · CM-e400e596 · confidence MEDIUM
```
return await db.reviewsCollection.find({ $where: 'this.product == ' + productId }) as Review[]
```

Why it matters: Request objects are passed directly to a database query, allowing operator injection ($ne, $gt, $where).

Recommended fix: Validate and cast types explicitly; reject objects where scalars are expected.

Verification plan: Send {"$ne": null} as a credential field in a test DB and confirm authentication bypass.

[HIGH] NoSQL / query operator injection · CWE-943 · A03:2021 Injection

data/static/codefixes/chatbotGreedyInjectionChallenge_4.ts:66 · CM-12309412 · confidence MEDIUM
```
return await db.reviewsCollection.find({ $where: 'this.product == ' + productId }) as Review[]
```

Why it matters: Request objects are passed directly to a database query, allowing operator injection ($ne, $gt, $where).

Recommended fix: Validate and cast types explicitly; reject objects where scalars are expected.

Verification plan: Send {"$ne": null} as a credential field in a test DB and confirm authentication bypass.

[HIGH] Hard-coded credential / API key · CWE-798 · A07:2021 Identification and Authentication

Failures

data/static/users.yml:3 · CM-fc1f8c86 · confidence MEDIUM
```
password: 'admin123'
```

Why it matters: A secret is embedded in source and will leak via version control, logs and build artifacts.

Recommended fix: Rotate the secret immediately; load from Secret Manager / environment at runtime.

Verification plan: Confirm the value is live using a read-only identity check in a controlled setting, then rotate.

[HIGH] Hard-coded credential / API key · CWE-798 · A07:2021 Identification and Authentication

Failures

data/static/users.yml:31 · CM-a332d6d8 · confidence MEDIUM
```
password: 'ncc-1701'
```

Why it matters: A secret is embedded in source and will leak via version control, logs and build artifacts.

Recommended fix: Rotate the secret immediately; load from Secret Manager / environment at runtime.

Verification plan: Confirm the value is live using a read-only identity check in a controlled setting, then rotate.

[HIGH] Hard-coded credential / API key · CWE-798 · A07:2021 Identification and Authentication

Failures

data/static/users.yml:63 · CM-9f9f7167 · confidence MEDIUM
```
password: 'OhG0dPlease1nsertLiquor!'
```

Why it matters: A secret is embedded in source and will leak via version control, logs and build artifacts.

Recommended fix: Rotate the secret immediately; load from Secret Manager / environment at runtime.

Verification plan: Confirm the value is live using a read-only identity check in a controlled setting, then rotate.

[HIGH] Hard-coded credential / API key · CWE-798 · A07:2021 Identification and Authentication

Failures

data/static/users.yml:88 · CM-7643de05 · confidence MEDIUM
```
password: 'bW9jLmxpYW1nQGhjaW5pbW1pay5ucmVvamI='
```

Why it matters: A secret is embedded in source and will leak via version control, logs and build artifacts.

Recommended fix: Rotate the secret immediately; load from Secret Manager / environment at runtime.

Verification plan: Confirm the value is live using a read-only identity check in a controlled setting, then rotate.

[HIGH] Hard-coded credential / API key · CWE-798 · A07:2021 Identification and Authentication Failures

data/static/users.yml:107 · CM-98bde1d7 · confidence MEDIUM
```
password: 'mDLx?94T~1CfVfZMzw@sJ9f?s3L6lbMqE70FfI8^54jbNikY5fymx7c!YbJb'
```

Why it matters: A secret is embedded in source and will leak via version control, logs and build artifacts.

Recommended fix: Rotate the secret immediately; load from Secret Manager / environment at runtime.

Verification plan: Confirm the value is live using a read-only identity check in a controlled setting, then rotate.

[HIGH] Hard-coded credential / API key · CWE-798 · A07:2021 Identification and Authentication Failures

data/static/users.yml:115 · CM-087b2fee · confidence MEDIUM
```
password: 'J6aVjTgOpRs@?5l!Zkq2AYnCE@RF$P'
```

Why it matters: A secret is embedded in source and will leak via version control, logs and build artifacts.

Recommended fix: Rotate the secret immediately; load from Secret Manager / environment at runtime.

Verification plan: Confirm the value is live using a read-only identity check in a controlled setting, then rotate.

[HIGH] Hard-coded credential / API key · CWE-798 · A07:2021 Identification and Authentication Failures

data/static/users.yml:123 · CM-d65f2f02 · confidence MEDIUM
```
password: 'focusOnScienceMorty!focusOnScience'
```

Why it matters: A secret is embedded in source and will leak via version control, logs and build artifacts.

Recommended fix: Rotate the secret immediately; load from Secret Manager / environment at runtime.

Verification plan: Confirm the value is live using a read-only identity check in a controlled setting, then rotate.

[HIGH] Hard-coded credential / API key · CWE-798 · A07:2021 Identification and Authentication Failures

data/static/users.yml:141 · CM-5e690dc0 · confidence MEDIUM
```
password: '0Y8rMnww$*9VFYE§59-!Fg1L6t&6lB'
```

Why it matters: A secret is embedded in source and will leak via version control, logs and build artifacts.

Recommended fix: Rotate the secret immediately; load from Secret Manager / environment at runtime.

Verification plan: Confirm the value is live using a read-only identity check in a controlled setting, then rotate.

[HIGH] Hard-coded credential / API key · CWE-798 · A07:2021 Identification and Authentication Failures

data/static/users.yml:150 · CM-ad186add · confidence MEDIUM
```
password: 'EinBelegtesBrotMitSchinkenSCHINKEN!'
```

Why it matters: A secret is embedded in source and will leak via version control, logs and build artifacts.

Recommended fix: Rotate the secret immediately; load from Secret Manager / environment at runtime.

Verification plan: Confirm the value is live using a read-only identity check in a controlled setting, then rotate.

[HIGH] Hard-coded credential / API key · CWE-798 · A07:2021 Identification and Authentication Failures

data/static/users.yml:159 · CM-eb0ede55 · confidence MEDIUM
```
password: 'K1f..................'
```

Why it matters: A secret is embedded in source and will leak via version control, logs and build artifacts.

Recommended fix: Rotate the secret immediately; load from Secret Manager / environment at runtime.

Verification plan: Confirm the value is live using a read-only identity check in a controlled setting, then rotate.

[HIGH] Hard-coded credential / API key · CWE-798 · A07:2021 Identification and Authentication Failures

data/static/users.yml:239 · CM-9c40940e · confidence MEDIUM
```
password: 'y&x5Z#f6W532Z4445#Ae2HkwZVyDb7&oCUaDzFU'
```

Why it matters: A secret is embedded in source and will leak via version control, logs and build artifacts.

Recommended fix: Rotate the secret immediately; load from Secret Manager / environment at runtime.

Verification plan: Confirm the value is live using a read-only identity check in a controlled setting, then rotate.

[HIGH] Hard-coded credential / API key · CWE-798 · A07:2021 Identification and Authentication Failures

data/static/users.yml:245 · CM-5154ca0b · confidence MEDIUM
```
password: 'y&x5Z#f6W532ZUf$q3DsdgfgfgxxUsvoCUaDzFU'
```

Why it matters: A secret is embedded in source and will leak via version control, logs and build artifacts.

Recommended fix: Rotate the secret immediately; load from Secret Manager / environment at runtime.

Verification plan: Confirm the value is live using a read-only identity check in a controlled setting, then rotate.

[HIGH] Hard-coded credential / API key · CWE-798 · A07:2021 Identification and Authentication Failures

data/static/users.yml:261 · CM-04335010 · confidence MEDIUM
```
password: 'private'
```

Why it matters: A secret is embedded in source and will leak via version control, logs and build artifacts.

Recommended fix: Rotate the secret immediately; load from Secret Manager / environment at runtime.

Verification plan: Confirm the value is live using a read-only identity check in a controlled setting, then rotate.

[HIGH] Hard-coded credential / API key · CWE-798 · A07:2021 Identification and Authentication Failures

data/static/users.yml:272 · CM-5b43abe4 · confidence MEDIUM
```
password: 'IamUsedForTesting'
```

Why it matters: A secret is embedded in source and will leak via version control, logs and build artifacts.

Recommended fix: Rotate the secret immediately; load from Secret Manager / environment at runtime.

Verification plan: Confirm the value is live using a read-only identity check in a controlled setting, then rotate.

[HIGH] Hard-coded credential / API key · CWE-798 · A07:2021 Identification and Authentication Failures

data/static/users.yml:277 · CM-7bfbfb15 · confidence MEDIUM
```
password: 'DVKH3gn7G$2Rk!f5Tz@9Lm&Qp4Wx#Yj8Bc0Ns6Ue1Io7Ah2Sd4Fg6Hj8Kl0Zx3Cv5Bn7Mq9Wr1Et3Yu5Ip'
```

Why it matters: A secret is embedded in source and will leak via version control, logs and build artifacts.

Recommended fix: Rotate the secret immediately; load from Secret Manager / environment at runtime.

Verification plan: Confirm the value is live using a read-only identity check in a controlled setting, then rotate.

[HIGH] Cross-site scripting (unsafe HTML sink) · CWE-79 · A03:2021 Injection

frontend/src/app/data-export/data-export.component.ts:72 · CM-9197a472 · confidence HIGH
```
window.open('', '_blank', 'width=500')?.document.write(this.userData)
```

Why it matters: Untrusted data reaches an HTML sink without contextual output encoding.

Recommended fix: Use textContent / auto-escaping templates and sanitise rich content with an allow-list sanitiser.

Verification plan: Submit a benign marker tag as input and confirm it is rendered unescaped in the response DOM.

[HIGH] Hard-coded credential / API key · CWE-798 · A07:2021 Identification and Authentication Failures

frontend/src/app/faucet/faucet.component.ts:34 · CM-25a30f61 · confidence MEDIUM
```
const BeeTokenAddress = '0x36435796Ca9be2bf150CE0dECc2D8Fab5C4d6E13'
```

Why it matters: A secret is embedded in source and will leak via version control, logs and build artifacts.

Recommended fix: Rotate the secret immediately; load from Secret Manager / environment at runtime.

Verification plan: Confirm the value is live using a read-only identity check in a controlled setting, then rotate.

[HIGH] Hard-coded credential / API key · CWE-798 · A07:2021 Identification and Authentication Failures

frontend/src/app/login/login.component.ts:69 · CM-ecf02202 · confidence MEDIUM
```
public testingPassword = 'IamUsedForTesting'
```

Why it matters: A secret is embedded in source and will leak via version control, logs and build artifacts.

Recommended fix: Rotate the secret immediately; load from Secret Manager / environment at runtime.

Verification plan: Confirm the value is live using a read-only identity check in a controlled setting, then rotate.

[HIGH] Cross-site scripting (unsafe HTML sink) · CWE-79 · A03:2021 Injection

frontend/src/assets/private/dat.gui.min.js:1 · CM-1de1e6a2 · confidence HIGH
```
var dat=dat||{};dat.gui=dat.gui||{};dat.utils=dat.utils||{};dat.controllers=dat.controllers||{};dat.dom=dat.dom||{};dat
```

Why it matters: Untrusted data reaches an HTML sink without contextual output encoding.

Recommended fix: Use textContent / auto-escaping templates and sanitise rich content with an allow-list sanitiser.

Verification plan: Submit a benign marker tag as input and confirm it is rendered unescaped in the response DOM.

Proposed patch (for human review):
```
--- a/frontend/src/assets/private/dat.gui.min.js
+++ b/frontend/src/assets/private/dat.gui.min.js
@@ -1,3 +1,3 @@
-var dat=dat||{};dat.gui=dat.gui||{};dat.utils=dat.utils||{};dat.controllers=dat.controllers||{};dat.dom=dat.d
+var dat=dat||{};dat.gui=dat.gui||{};dat.utils=dat.utils||{};dat.controllers=dat.controllers||{};dat.dom=dat.d
```

```
dat.utils.common=function(){var e=Array.prototype.forEach,a=Array.prototype.slice;return{BREAK:{},extend:func
    each:function(a,d,f){if(e&&a.forEach===e)a.forEach(d,f);else if(a.length===a.length+0)for(var c=0,p=a.len
```

[HIGH] Cross-site scripting (unsafe HTML sink) · CWE-79 · A03:2021 Injection

frontend/src/assets/private/dat.gui.min.js:13 · CM-437d5842 · confidence HIGH

```
dat.controllers.OptionController=function(e,a,b){var d=function(f,c,e){d.superclass.call(this,f,c);var k=this;this.__se
```

Why it matters: Untrusted data reaches an HTML sink without contextual output encoding.

Recommended fix: Use textContent / auto-escaping templates and sanitise rich content with an allow-list sanitiser.

Verification plan: Submit a benign marker tag as input and confirm it is rendered unescaped in the response DOM.

Proposed patch (for human review):

```
--- a/frontend/src/assets/private/dat.gui.min.js
+++ b/frontend/src/assets/private/dat.gui.min.js
@@ -11,5 +11,5 @@
         e&&(b.splice(e,1),a.className=b.join(" "))}}else a.className=void 0;return f},hasClass:function(a,d){
    {left:0,top:0};if(a.offsetParent){do d.left+=a.offsetLeft,d.top+=a.offsetTop;while(a=a.offsetParent)}retu
-dat.controllers.OptionController=function(e,a,b){var d=function(f,c,e){d.superclass.call(this,f,c);var k=this
+dat.controllers.OptionController=function(e,a,b){var d=function(f,c,e){d.superclass.call(this,f,c);var k=this
    d.superclass=e;b.extend(d.prototype,e.prototype,{setValue:function(a){a=d.superclass.prototype.setValue.c
 dat.controllers.NumberController=function(e,a){var b=function(d,f,c){b.superclass.call(this,d,f);c=c||{};this
```

[HIGH] Cross-site scripting (unsafe HTML sink) · CWE-79 · A03:2021 Injection

frontend/src/assets/private/dat.gui.min.js:23 · CM-7cef06e1 · confidence HIGH

```
dat.controllers.FunctionController=function(e,a,b){var d=function(b,c,e){d.superclass.call(this,b,c);var k=this;this.__
```

Why it matters: Untrusted data reaches an HTML sink without contextual output encoding.

Recommended fix: Use textContent / auto-escaping templates and sanitise rich content with an allow-list sanitiser.

Verification plan: Submit a benign marker tag as input and confirm it is rendered unescaped in the response DOM.

Proposed patch (for human review):

```
--- a/frontend/src/assets/private/dat.gui.min.js
+++ b/frontend/src/assets/private/dat.gui.min.js
@@ -21,5 +21,5 @@
     document.createElement("div");this.__foreground=document.createElement("div");a.bind(this.__background,"m
     (this.getValue()-this.__min)/(this.__max-this.__min);this.__foreground.style.width=100*a+"%";return p.sup
-dat.controllers.FunctionController=function(e,a,b){var d=function(b,c,e){d.superclass.call(this,b,c);var k=th
+dat.controllers.FunctionController=function(e,a,b){var d=function(b,c,e){d.superclass.call(this,b,c);var k=th
     this.getValue());this.getValue().call(this.object)}});return d}(dat.controllers.Controller,dat.dom.dom,da
 dat.controllers.BooleanController=function(e,a,b){var d=function(b,c){d.superclass.call(this,b,c);var e=this;
```

[HIGH] Cross-site scripting (unsafe HTML sink) · CWE-79 · A03:2021 Injection

frontend/src/assets/private/dat.gui.min.js:33 · CM-9cb42b4f · confidence HIGH

```
dat.GUI=dat.gui.GUI=function(e,a,b,d,f,c,p,k,l,q,n,r,y,g,h){function t(a,c,b,e){if(void 0===c[b])throw Error("Object "+
```

Why it matters: Untrusted data reaches an HTML sink without contextual output encoding.

Recommended fix: Use textContent / auto-escaping templates and sanitise rich content with an allow-list sanitiser.

Verification plan: Submit a benign marker tag as input and confirm it is rendered unescaped in the response DOM.

Proposed patch (for human review):

```
--- a/frontend/src/assets/private/dat.gui.min.js
+++ b/frontend/src/assets/private/dat.gui.min.js
@@ -31,5 +31,5 @@
     a.isNumber(c.g)&&a.isNumber(c.b)?{space:"RGB",r:c.r,g:c.g,b:c.b}:!1},write:function(a){return{r:a.r,g:a.g
     var c=1<arguments.length?a.toArray(arguments):arguments[0];a.each(f,function(e){if(e.litmus(c))return a.e
-dat.GUI=dat.gui.GUI=function(e,a,b,d,f,c,p,k,l,q,n,r,y,g,h){function t(a,c,b,e){if(void 0===c[b])throw Error(
```

```
+dat.GUI=dat.gui.GUI=function(e,a,b,d,f,c,p,k,l,q,n,r,y,g,h){function t(a,c,b,e){if(void 0===c[b])throw Error(
     g.addClass(e,typeof c.getValue());s(a,e,c);a.__controllers.push(c);return c}function u(a,d,c){var b=docum
     {before:b.__li.nextElementSibling,factoryArgs:[d]})},name:function(a){b.__li.firstElementChild.firstEleme
```

[HIGH] Cross-site scripting (unsafe HTML sink) · CWE-79 · A03:2021 Injection

frontend/src/assets/private/dat.gui.min.js:35 · CM-c8a83128 · confidence HIGH

```
{before:b.__li.nextElementSibling,factoryArgs:[d]})},name:function(a){b.__li.firstElementChild.firstElementChild.innerH
```

Why it matters: Untrusted data reaches an HTML sink without contextual output encoding.

Recommended fix: Use textContent / auto-escaping templates and sanitise rich content with an allow-list sanitiser.

Verification plan: Submit a benign marker tag as input and confirm it is rendered unescaped in the response DOM.

Proposed patch (for human review):

```
--- a/frontend/src/assets/private/dat.gui.min.js
+++ b/frontend/src/assets/private/dat.gui.min.js
@@ -33,5 +33,5 @@
 dat.GUI=dat.gui.GUI=function(e,a,b,d,f,c,p,k,l,q,n,r,y,g,h){function t(a,c,b,e){if(void 0===c[b])throw Error(
     g.addClass(e,typeof c.getValue());s(a,e,c);a.__controllers.push(c);return c}function u(a,d,c){var b=docum
-    {before:b.__li.nextElementSibling,factoryArgs:[d]})},name:function(a){b.__li.firstElementChild.firstEleme
+    {before:b.__li.nextElementSibling,factoryArgs:[d]})},name:function(a){b.__li.firstElementChild.firstEleme
     g.addClass(d,"has-slider");b.domElement.insertBefore(e.domElement,b.domElement.firstElementChild)}else if
     b instanceof p?(g.bind(d,"click",function(){g.fakeEvent(b.__button,"click")}),g.bind(d,"mouseover",functi
```

[HIGH] Cross-site scripting (unsafe HTML sink) · CWE-79 · A03:2021 Injection

frontend/src/assets/private/dat.gui.min.js:39 · CM-1652ac93 · confidence HIGH

```
a.__ul.firstChild);g.addClass(b,"save-row");var d=document.createElement("span");d.innerHTML=" ";g.addClass(d,"but
```

Why it matters: Untrusted data reaches an HTML sink without contextual output encoding.

Recommended fix: Use textContent / auto-escaping templates and sanitise rich content with an allow-list sanitiser.

Verification plan: Submit a benign marker tag as input and confirm it is rendered unescaped in the response DOM.

Proposed patch (for human review):

```
--- a/frontend/src/assets/private/dat.gui.min.js
+++ b/frontend/src/assets/private/dat.gui.min.js
@@ -37,5 +37,5 @@
     b instanceof p?(g.bind(d,"click",function(){g.fakeEvent(b.__button,"click")}),g.bind(d,"mouseover",functi

-    a.__ul.firstChild);g.addClass(b,"save-row");var d=document.createElement("span");d.innerHTML=" ";g.a
+    a.__ul.firstChild);g.addClass(b,"save-row");var d=document.createElement("span");d.textContent =" ";
     a.load.remembered?h.each(a.load.remembered,function(b,d){E(a,d,d==a.preset)}):E(a,z,!1);g.bind(q,"change"
     localStorage.getItem(document.location.href+".isLocal")&&b.setAttribute("checked","checked");var k=functi
```

[HIGH] Cross-site scripting (unsafe HTML sink) · CWE-79 · A03:2021 Injection

frontend/src/assets/private/dat.gui.min.js:40 · CM-9bf87dea · confidence HIGH

```
a.load.remembered?h.each(a.load.remembered,function(b,d){E(a,d,d==a.preset)}):E(a,z,!1);g.bind(q,"change",function(){fo
```

Why it matters: Untrusted data reaches an HTML sink without contextual output encoding.

Recommended fix: Use textContent / auto-escaping templates and sanitise rich content with an allow-list sanitiser.

Verification plan: Submit a benign marker tag as input and confirm it is rendered unescaped in the response DOM.

Proposed patch (for human review):

```
--- a/frontend/src/assets/private/dat.gui.min.js
+++ b/frontend/src/assets/private/dat.gui.min.js
@@ -38,5 +38,5 @@

     a.__ul.firstChild);g.addClass(b,"save-row");var d=document.createElement("span");d.innerHTML=" ";g.a
```

```
-    a.load.remembered?h.each(a.load.remembered,function(b,d){E(a,d,d==a.preset)}):E(a,z,!1);g.bind(q,"change"
+    a.load.remembered?h.each(a.load.remembered,function(b,d){E(a,d,d==a.preset)}):E(a,z,!1);g.bind(q,"change"
     localStorage.getItem(document.location.href+".isLocal")&&b.setAttribute("checked","checked");var k=functi
     function(){a.save()});g.bind(e,"click",function(){var b=prompt("Enter a new preset name.");b&&a.saveAs(b)
```

[HIGH] Cross-site scripting (unsafe HTML sink) · CWE-79 · A03:2021 Injection

frontend/src/assets/private/dat.gui.min.js:41 · CM-e72e9ced · confidence HIGH

```
localStorage.getItem(document.location.href+".isLocal")&&b.setAttribute("checked","checked");var k=function(){n.style.d
```

Why it matters: Untrusted data reaches an HTML sink without contextual output encoding.

Recommended fix: Use textContent / auto-escaping templates and sanitise rich content with an allow-list sanitiser.

Verification plan: Submit a benign marker tag as input and confirm it is rendered unescaped in the response DOM.

Proposed patch (for human review):

```
--- a/frontend/src/assets/private/dat.gui.min.js
+++ b/frontend/src/assets/private/dat.gui.min.js
@@ -39,5 +39,5 @@
     a.__ul.firstChild);g.addClass(b,"save-row");var d=document.createElement("span");d.innerHTML=" ";g.a
     a.load.remembered?h.each(a.load.remembered,function(b,d){E(a,d,d==a.preset)}):E(a,z,!1);g.bind(q,"change"
-    localStorage.getItem(document.location.href+".isLocal")&&b.setAttribute("checked","checked");var k=functi
+    localStorage.getItem(document.location.href+".isLocal")&&b.setAttribute("checked","checked");var k=functi
     function(){a.save()});g.bind(e,"click",function(){var b=prompt("Enter a new preset name.");b&&a.saveAs(b)
     "mouseup",c)}a.__resize_handle=document.createElement("div");h.extend(a.__resize_handle.style,{width:"6px
```

[HIGH] Cross-site scripting (unsafe HTML sink) · CWE-79 · A03:2021 Injection

frontend/src/assets/private/dat.gui.min.js:44 · CM-1bba65f6 · confidence HIGH

```
function B(a,b){var d={};h.each(a.__rememberedObjects,function(c,e){var f={};h.each(a.__rememberedObjectIndecesToContro
```

Why it matters: Untrusted data reaches an HTML sink without contextual output encoding.

Recommended fix: Use textContent / auto-escaping templates and sanitise rich content with an allow-list sanitiser.

Verification plan: Submit a benign marker tag as input and confirm it is rendered unescaped in the response DOM.

Proposed patch (for human review):

```
--- a/frontend/src/assets/private/dat.gui.min.js
+++ b/frontend/src/assets/private/dat.gui.min.js
@@ -42,5 +42,5 @@
     function(){a.save()});g.bind(e,"click",function(){var b=prompt("Enter a new preset name.");b&&a.saveAs(b)
     "mouseup",c)}a.__resize_handle=document.createElement("div");h.extend(a.__resize_handle.style,{width:"6px
-    function B(a,b){var d={};h.each(a.__rememberedObjects,function(c,e){var f={};h.each(a.__rememberedObjectI
+    function B(a,b){var d={};h.each(a.__rememberedObjects,function(c,e){var f={};h.each(a.__rememberedObjectI
     r(function(){G(a)});h.each(a,function(a){a.updateDisplay()})}e.inject(b);var z="Default",w;try{w="localSt
         g.addClass(this.domElement,"dg");this.__folders={};this.__controllers=[];this.__rememberedObjects=[];
```

[HIGH] Cross-site scripting (unsafe HTML sink) · CWE-79 · A03:2021 Injection

frontend/src/assets/private/dat.gui.min.js:48 · CM-84b26eee · confidence HIGH

```
b);c.revert()}},width:{get:function(){return a.width},set:function(b){a.width=b;F(c,b)}},name:{get:function(){return a.
```

Why it matters: Untrusted data reaches an HTML sink without contextual output encoding.

Recommended fix: Use textContent / auto-escaping templates and sanitise rich content with an allow-list sanitiser.

Verification plan: Submit a benign marker tag as input and confirm it is rendered unescaped in the response DOM.

Proposed patch (for human review):

```
--- a/frontend/src/assets/private/dat.gui.min.js
+++ b/frontend/src/assets/private/dat.gui.min.js
@@ -46,5 +46,5 @@
         g.addClass(this.domElement,"dg");this.__folders={};this.__controllers=[];this.__rememberedObjects=[];
```

```
         (a.scrollable=!0);var e=w&&"true"===localStorage.getItem(document.location.href+".isLocal");Object.de
-            b);c.revert()}},width:{get:function(){return a.width},set:function(b){a.width=b;F(c,b)}},name:{ge
+            b);c.revert()}},width:{get:function(){return a.width},set:function(b){a.width=b;F(c,b)}},name:{ge
         ((e=a)?g.bind(window,"unload",b):g.unbind(window,"unload",b),localStorage.setItem(document.location.h
             g.bind(this.__closeButton,"click",function(){c.closed=!c.closed})}else{void 0===a.closed&&(a.clos
```

[HIGH] Cross-site scripting (unsafe HTML sink) · CWE-79 · A03:2021 Injection

frontend/src/assets/private/dat.gui.min.js:49 · CM-d5d059de · confidence HIGH

```
((e=a)?g.bind(window,"unload",b):g.unbind(window,"unload",b),localStorage.setItem(document.location.href+".isLocal",a))
```

Why it matters: Untrusted data reaches an HTML sink without contextual output encoding.

Recommended fix: Use textContent / auto-escaping templates and sanitise rich content with an allow-list sanitiser.

Verification plan: Submit a benign marker tag as input and confirm it is rendered unescaped in the response DOM.

Proposed patch (for human review):

```
--- a/frontend/src/assets/private/dat.gui.min.js
+++ b/frontend/src/assets/private/dat.gui.min.js
@@ -47,5 +47,5 @@
         (a.scrollable=!0);var e=w&&"true"===localStorage.getItem(document.location.href+".isLocal");Object.de
             b);c.revert()}},width:{get:function(){return a.width},set:function(b){a.width=b;F(c,b)}},name:{ge
-        ((e=a)?g.bind(window,"unload",b):g.unbind(window,"unload",b),localStorage.setItem(document.location.h
+        ((e=a)?g.bind(window,"unload",b):g.unbind(window,"unload",b),localStorage.setItem(document.location.h
             g.bind(this.__closeButton,"click",function(){c.closed=!c.closed})}else{void 0===a.closed&&(a.clos
             H=!1),x.appendChild(this.domElement),g.addClass(this.domElement,m.CLASS_AUTO_PLACE)),this.parent|
```

[HIGH] Cross-site scripting (unsafe HTML sink) · CWE-79 · A03:2021 Injection

frontend/src/assets/private/dat.gui.min.js:55 · CM-765d9f9a · confidence HIGH

```
b-20+"px"):(g.removeClass(a.domElement,m.CLASS_TOO_TALL),a.__ul.style.height="auto")}a.__resize_handle&&h.defer(functio
```

Why it matters: Untrusted data reaches an HTML sink without contextual output encoding.

Recommended fix: Use textContent / auto-escaping templates and sanitise rich content with an allow-list sanitiser.

Verification plan: Submit a benign marker tag as input and confirm it is rendered unescaped in the response DOM.

Proposed patch (for human review):

```
--- a/frontend/src/assets/private/dat.gui.min.js
+++ b/frontend/src/assets/private/dat.gui.min.js
@@ -53,5 +53,5 @@
         2)})},addColor:function(a,b){return t(this,a,b,{color:!0})},remove:function(a){this.__ul.removeChild(
     (b.closed=this.load.folders[a].closed,b.load=this.load.folders[a]);b=new m(b);this.__folders[a]=b;a=u(thi
-        b-20+"px"):(g.removeClass(a.domElement,m.CLASS_TOO_TALL),a.__ul.style.height="auto")}a.__resize_handl
+        b-20+"px"):(g.removeClass(a.domElement,m.CLASS_TOO_TALL),a.__ul.style.height="auto")}a.__resize_handl
         b.__rememberedObjects.indexOf(a)&&b.__rememberedObjects.push(a)});this.autoPlace&&F(this,this.width)}
     {});this.load.remembered[this.preset]=B(this);D(this,!1)},saveAs:function(a){this.load.remembered||(this.
```

[HIGH] Cross-site scripting (unsafe HTML sink) · CWE-79 · A03:2021 Injection

frontend/src/assets/private/stats.min.js:3 · CM-793c7087 · confidence HIGH

```
i.innerHTML="FPS";a.appendChild(i);var c=document.createElement("div");c.id="fpsGraph";c.style.cssText="position:relati
```

Why it matters: Untrusted data reaches an HTML sink without contextual output encoding.

Recommended fix: Use textContent / auto-escaping templates and sanitise rich content with an allow-list sanitiser.

Verification plan: Submit a benign marker tag as input and confirm it is rendered unescaped in the response DOM.

Proposed patch (for human review):

```
--- a/frontend/src/assets/private/stats.min.js
+++ b/frontend/src/assets/private/stats.min.js
@@ -1,5 +1,5 @@
```

```
 // stats.js - http://github.com/mrdoob/stats.js
 var Stats=function(){var l=Date.now(),m=l,g=0,n=Infinity,o=0,h=0,p=Infinity,q=0,r=0,s=0,f=document.createElem
-    i.innerHTML="FPS";a.appendChild(i);var c=document.createElement("div");c.id="fpsGraph";c.style.cssText="p
+    i.textContent ="FPS";a.appendChild(i);var c=document.createElement("div");c.id="fpsGraph";c.style.cssText
     k.id="msText";k.style.cssText="color:#0f0;font-family:Helvetica,Arial,sans-serif;font-size:9px;font-weigh
         "block";d.style.display="none";break;case 1:a.style.display="none",d.style.display="block"}};return{R
```

[HIGH] Cross-site scripting (unsafe HTML sink) · CWE-79 · A03:2021 Injection

frontend/src/assets/private/stats.min.js:4 · CM-374c7e1b · confidence HIGH

```
k.id="msText";k.style.cssText="color:#0f0;font-family:Helvetica,Arial,sans-serif;font-size:9px;font-weight:bold;line-he
```

Why it matters: Untrusted data reaches an HTML sink without contextual output encoding.

Recommended fix: Use textContent / auto-escaping templates and sanitise rich content with an allow-list sanitiser.

Verification plan: Submit a benign marker tag as input and confirm it is rendered unescaped in the response DOM.

Proposed patch (for human review):

```
--- a/frontend/src/assets/private/stats.min.js
+++ b/frontend/src/assets/private/stats.min.js
@@ -2,5 +2,5 @@
 var Stats=function(){var l=Date.now(),m=l,g=0,n=Infinity,o=0,h=0,p=Infinity,q=0,r=0,s=0,f=document.createElem
     i.innerHTML="FPS";a.appendChild(i);var c=document.createElement("div");c.id="fpsGraph";c.style.cssText="p
-    k.id="msText";k.style.cssText="color:#0f0;font-family:Helvetica,Arial,sans-serif;font-size:9px;font-weigh
+    k.id="msText";k.style.cssText="color:#0f0;font-family:Helvetica,Arial,sans-serif;font-size:9px;font-weigh
         "block";d.style.display="none";break;case 1:a.style.display="none",d.style.display="block"}};return{R
         a+"px",m=b,r=0);return b},update:function(){l=this.end()}}};
```

[HIGH] Cross-site scripting (unsafe HTML sink) · CWE-79 · A03:2021 Injection

frontend/src/assets/private/three.js:11354 · CM-91f910d4 · confidence HIGH

```
e.innerHTML = "Loading ...";
```

Why it matters: Untrusted data reaches an HTML sink without contextual output encoding.

Recommended fix: Use textContent / auto-escaping templates and sanitise rich content with an allow-list sanitiser.

Verification plan: Submit a benign marker tag as input and confirm it is rendered unescaped in the response DOM.

Proposed patch (for human review):

```
--- a/frontend/src/assets/private/three.js
+++ b/frontend/src/assets/private/three.js
@@ -11352,5 +11352,5 @@
  ■■e.style.zIndex = 1000;

-■■e.innerHTML = "Loading ...";
+■■e.textContent = "Loading ...";

  ■■return e;
```

[HIGH] Cross-site scripting (unsafe HTML sink) · CWE-79 · A03:2021 Injection

frontend/src/assets/private/three.js:11375 · CM-8d16daed · confidence HIGH

```
this.statusDomElement.innerHTML = message;
```

Why it matters: Untrusted data reaches an HTML sink without contextual output encoding.

Recommended fix: Use textContent / auto-escaping templates and sanitise rich content with an allow-list sanitiser.

Verification plan: Submit a benign marker tag as input and confirm it is rendered unescaped in the response DOM.

Proposed patch (for human review):

```
--- a/frontend/src/assets/private/three.js
+++ b/frontend/src/assets/private/three.js
```

```
@@ -11373,5 +11373,5 @@
 ■■}

-■■this.statusDomElement.innerHTML = message;
+■■this.statusDomElement.textContent = message;

 ■},
```

[HIGH] Cross-site scripting (unsafe HTML sink) · CWE-79 · A03:2021 Injection

frontend/src/hacking-instructor/helpers/helpers.ts:138 · CM-37c445b6 · confidence HIGH

```
if (element && element.innerHTML === value) {
```

Why it matters: Untrusted data reaches an HTML sink without contextual output encoding.

Recommended fix: Use textContent / auto-escaping templates and sanitise rich content with an allow-list sanitiser.

Verification plan: Submit a benign marker tag as input and confirm it is rendered unescaped in the response DOM.

Proposed patch (for human review):

```
--- a/frontend/src/hacking-instructor/helpers/helpers.ts
+++ b/frontend/src/hacking-instructor/helpers/helpers.ts
@@ -136,5 +136,5 @@
       )

-      if (element && element.innerHTML === value) {
+      if (element && element.textContent === value) {
         break
       }
```

[HIGH] Cross-site scripting (unsafe HTML sink) · CWE-79 · A03:2021 Injection

frontend/src/hacking-instructor/index.ts:126 · CM-8397a5f0 · confidence HIGH

```
textBox.innerHTML = snarkdown(hint.text)
```

Why it matters: Untrusted data reaches an HTML sink without contextual output encoding.

Recommended fix: Use textContent / auto-escaping templates and sanitise rich content with an allow-list sanitiser.

Verification plan: Submit a benign marker tag as input and confirm it is rendered unescaped in the response DOM.

Proposed patch (for human review):

```
--- a/frontend/src/hacking-instructor/index.ts
+++ b/frontend/src/hacking-instructor/index.ts
@@ -124,5 +124,5 @@

   const textBox = createElement('span', { flexGrow: '2' })
-  textBox.innerHTML = snarkdown(hint.text)
+  textBox.textContent = snarkdown(hint.text)

   const cancelButtonStyles = {
```

[HIGH] Cross-site scripting (unsafe HTML sink) · CWE-79 · A03:2021 Injection

frontend/src/hacking-instructor/index.ts:142 · CM-b69f5120 · confidence HIGH

```
cancelButton.innerHTML = '<div>&times;</div>'
```

Why it matters: Untrusted data reaches an HTML sink without contextual output encoding.

Recommended fix: Use textContent / auto-escaping templates and sanitise rich content with an allow-list sanitiser.

Verification plan: Submit a benign marker tag as input and confirm it is rendered unescaped in the response DOM.

Proposed patch (for human review):

```
--- a/frontend/src/hacking-instructor/index.ts
```

```
+++ b/frontend/src/hacking-instructor/index.ts
@@ -140,5 +140,5 @@

   const cancelButton = createElement('button', cancelButtonStyles, { id: 'cancelButton', title: 'Cancel the t
-  cancelButton.innerHTML = '<div>&times;</div>'
+  cancelButton.textContent = '<div>&times;</div>'

   elem.appendChild(picture)
```

[HIGH] Hard-coded credential / API key · CWE-798 · A07:2021 Identification and Authentication

Failures

infrastructure/terraform/networking.tf:171 · CM-57da77f4 · confidence MEDIUM
```
private_key      = "-----BEGIN RSA PRIVATE KEY-----\nMIICXAIBAAKBgQDNwqLEe9wgTXCbC7+RPdDbBbeqjdbs4kOPOIGzqLpXvJXlxxW8\n
```

Why it matters: A secret is embedded in source and will leak via version control, logs and build artifacts.

Recommended fix: Rotate the secret immediately; load from Secret Manager / environment at runtime.

Verification plan: Confirm the value is live using a read-only identity check in a controlled setting, then rotate.

[HIGH] Hard-coded credential / API key · CWE-798 · A07:2021 Identification and Authentication

Failures

lib/insecurity.ts:21 · CM-a292874b · confidence MEDIUM
```
const privateKey = '-----BEGIN RSA PRIVATE KEY-----\r\nMIICXAIBAAKBgQDNwqLEe9wgTXCbC7+RPdDbBbeqjdbs4kOPOIGzqLpXvJXlxxW8
```

Why it matters: A secret is embedded in source and will leak via version control, logs and build artifacts.

Recommended fix: Rotate the secret immediately; load from Secret Manager / environment at runtime.

Verification plan: Confirm the value is live using a read-only identity check in a controlled setting, then rotate.

[HIGH] Code injection via eval/exec · CWE-95 · A03:2021 Injection

routes/captcha.ts:22 · CM-85868b16 · confidence MEDIUM
```
const answer = eval(expression).toString() // eslint-disable-line no-eval
```

Why it matters: Dynamic evaluation of strings can execute attacker-controlled code.

Recommended fix: Remove eval/exec; use ast.literal_eval or JSON parsing for data.

Verification plan: Feed an expression with an observable side-effect marker in a sandbox and confirm evaluation.

[HIGH] NoSQL / query operator injection · CWE-943 · A03:2021 Injection

routes/chat.ts:149 · CM-933f1b2a · confidence MEDIUM
```
return await db.reviewsCollection.find({ $where: 'this.product == ' + productId }) as Review[]
```

Why it matters: Request objects are passed directly to a database query, allowing operator injection ($ne, $gt, $where).

Recommended fix: Validate and cast types explicitly; reject objects where scalars are expected.

Verification plan: Send {"$ne": null} as a credential field in a test DB and confirm authentication bypass.

[HIGH] Path traversal in file access · CWE-22 · A01:2021 Broken Access Control

routes/languages.ts:31 · CM-8a7e6bc3 · confidence MEDIUM
```
const content = await readFile('frontend/dist/frontend/assets/i18n/' + fileName, 'utf-8')
```

Why it matters: A file path is derived from request input without canonicalisation or base-directory checks.

Recommended fix: Resolve the path, verify it stays under an allowed base directory, and use an ID-to-file mapping.

Verification plan: Request a path containing parent-directory segments against a test file tree and confirm a file outside the base is returned.

[HIGH] Path traversal in file access · CWE-22 · A01:2021 Broken Access Control

routes/languages.ts:40 · CM-ba6c8c25 · confidence MEDIUM

```
const backendContent = await readFile('i18n/' + fileName, 'utf-8')
```

Why it matters: A file path is derived from request input without canonicalisation or base-directory checks.

Recommended fix: Resolve the path, verify it stays under an allowed base directory, and use an ID-to-file mapping.

Verification plan: Request a path containing parent-directory segments against a test file tree and confirm a file outside the base is returned.

47 lower-severity findings are listed in the overview only.

Method and limits

Findings come from a built-in rule-based scanner mapped to CWE and OWASP, not from the CodeMender product. Verification plans describe how a finding would be confirmed; no exploit was executed. Patches are suggestions that must be reviewed and tested before merge. Test folders and test files (test, tests, spec, e2e, cypress, fixtures, *.spec.*, *.test.*) and vendored or build folders are not scanned.